JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class FileInputStream

java.lang.Object
    java.io.InputStream
        java.io.FileInputStream

All Implemented Interfaces:

Closeable, AutoCloseable

```
public class FileInputStream
extends InputStream
```

A `FileInputStream` obtains input bytes from a file in a file system. What files are available depends on the host environment.

`FileInputStream` is meant for reading streams of raw bytes such as image data. For reading streams of characters, consider using `FileReader`.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the `try`-with-resources statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use `Cleaner` or some other mechanism.

Since:

1.0

See Also:

File,
FileDescriptor,
FileOutputStream,
Files.newInputStream(Path, OpenOption...)

Constructor Summary

Constructors

Constructor	Description
FileInputStream(File file)	Creates a FileInputStream to read from an existing file represented by the File object file.
FileInputStream(FileDescriptor fdObj)	Creates a FileInputStream by using the file descriptor fdObj, which

	represents an existing connection to an actual file in the file system.
FileInputStream(String name)	Creates a FileInputStream to read from an existing file named by the path name name.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
int	**available**()	Returns an estimate of the number of remaining bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
void	**close**()	Closes this file input stream and releases any system resources associated with the stream.
FileChannel	**getChannel**()	Returns the unique FileChannel object associated with this file input stream.
final FileDescriptor	**getFD**()	Returns the FileDescriptor object that represents the connection to the actual file in the file system being used by this FileInputStream.
int	**read**()	

		Reads a byte of data from this input stream.
int	**read**(byte[] b)	Reads up to b.length bytes of data from this input stream into an array of bytes.
int	**read**(byte[] b, int off, int len)	Reads up to len bytes of data from this input stream into an array of bytes.
byte[]	**readNBytes**(int len)	Reads up to a specified number of bytes from the input stream.
long	**skip**(long n)	Skips over and discards n bytes of data from the input stream.

Methods declared in class InputStream

mark, markSupported, nullInputStream, readAllBytes, readNBytes, reset, skipNBytes, transferTo

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

FileInputStream

```
public FileInputStream(String name)
                throws FileNotFoundException
```

Creates a FileInputStream to read from an existing file named by the path name name. Symbolic links are automatically redirected to the *target* of the link. A new FileDescriptor object is created to represent this file connection.

If the named file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading then a FileNotFoundException is thrown.

Parameters:

`name` - the system-dependent file name.

Throws:

`FileNotFoundException` - if the file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading.

FileInputStream

```
public FileInputStream(File file)
                throws FileNotFoundException
```

Creates a `FileInputStream` to read from an existing file represented by the `File` object `file`. Symbolic links are automatically redirected to the *target* of the link. A new `FileDescriptor` object is created to represent this file connection.

If the named file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading then a `FileNotFoundException` is thrown.

Parameters:

`file` - the file to be opened for reading.

Throws:

`FileNotFoundException` - if the file does not exist, is a directory rather than a regular file, or for some other reason cannot be opened for reading.

See Also:

`File.getPath()`

FileInputStream

```
public FileInputStream(FileDescriptor fdObj)
```

Creates a `FileInputStream` by using the file descriptor `fdObj`, which represents an existing connection to an actual file in the file system.

If `fdObj` is null then a `NullPointerException` is thrown.

This constructor does not throw an exception if `fdObj` is `invalid`. However, if the methods are invoked on the resulting stream to attempt I/O on the stream, an `IOException` is thrown.

Parameters:

`fdObj` - the file descriptor to be opened for reading.

Method Details

read

```
public int read()
         throws IOException
```

Reads a byte of data from this input stream. This method blocks if no input is yet available.

Specified by:

`read` in class `InputStream`

Returns:

the next byte of data, or `-1` if the end of the file is reached.

Throws:

`IOException` - if an I/O error occurs.

read

```
public int read(byte[] b)
         throws IOException
```

Reads up to `b.length` bytes of data from this input stream into an array of bytes. This method blocks until some input is available.

Overrides:

`read` in class `InputStream`

Parameters:

`b` - the buffer into which the data is read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the file has been reached.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`InputStream.read(byte[], int, int)`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads up to `len` bytes of data from this input stream into an array of bytes. If `len` is not zero, the method blocks until some input is available; otherwise, no bytes are read and `0` is returned.

Overrides:

`read` in class `InputStream`

Parameters:

`b` - the buffer into which the data is read.

`off` - the start offset in array `b` at which the data is written.

`len` - the maximum number of bytes to read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`NullPointerException` - If `b` is `null`.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

`IOException` - if an I/O error occurs.

See Also:

`InputStream.read()`

readNBytes

```
public byte[] readNBytes(int len)
                  throws IOException
```

Description copied from class: `InputStream`

Reads up to a specified number of bytes from the input stream. This method blocks until the requested number of bytes has been read, end of stream is detected, or an exception is thrown. This method does not close the input stream.

The length of the returned array equals the number of bytes read from the stream. If `len` is zero, then no bytes are read and an empty byte array is returned. Otherwise, up to `len` bytes are read from the stream. Fewer than `len` bytes may be read if end of stream is encountered.

When this stream reaches end of stream, further invocations of this method will return an empty byte array.

Note that this method is intended for simple cases where it is convenient to read the specified number of bytes into a byte array. The total amount of memory allocated by this method is proportional to the number of bytes read

from the stream which is bounded by `len`. Therefore, the method may be safely called with very large values of `len` provided sufficient memory is available.

The behavior for the case where the input stream is *asynchronously closed*, or the thread interrupted during the read, is highly input stream specific, and therefore not specified.

If an I/O error occurs reading from the input stream, then it may do so after some, but not all, bytes have been read. Consequently the input stream may not be at end of stream and may be in an inconsistent state. It is strongly recommended that the stream be promptly closed if an I/O error occurs.

Overrides:

`readNBytes` in class `InputStream`

Parameters:

`len` - the maximum number of bytes to read

Returns:

a byte array containing the bytes read from this input stream

Throws:

`IOException` - if an I/O error occurs

Since:

11

skip

```
public long skip(long n)
          throws IOException
```

Skips over and discards `n` bytes of data from the input stream.

The `skip` method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly `0`. If `n` is negative, the method will try to skip backwards. In case the backing file does not support backward skip at its current position, an `IOException` is thrown. The actual number of bytes skipped is returned. If it skips forwards, it returns a positive value. If it skips backwards, it returns a negative value.

This method may skip more bytes than what are remaining in the backing file. This produces no exception and the number of bytes skipped may include some number of bytes that were beyond the EOF of the backing file. Attempting to read from the stream after skipping past the end will result in -1 indicating the end of the file.

Overrides:

skip in class InputStream

Parameters:

n - the number of bytes to be skipped.

Returns:

the actual number of bytes skipped.

Throws:

IOException - if n is negative, if the stream does not support seek, or if an I/O error occurs.

See Also:

InputStream.skipNBytes(long)

available

```
public int available()
             throws IOException
```

Returns an estimate of the number of remaining bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream. Returns 0 when the file position is beyond EOF. The next invocation might be the same thread or another thread. A single read or skip of this many bytes will not block, but may read or skip fewer bytes.

In some cases, a non-blocking read (or skip) may appear to be blocked when it is merely slow, for example when reading large files over slow networks.

Overrides:

available in class InputStream

Returns:

an estimate of the number of remaining bytes that can be read (or skipped over) from this input stream without blocking.

Throws:

IOException - if this file input stream has been closed by calling `close` or an I/O error occurs.

close

```
public void close()
           throws IOException
```

Closes this file input stream and releases any system resources associated with the stream.

If this stream has an associated channel then the channel is closed as well.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `InputStream`

API Note:

Overriding `close()` to perform cleanup actions is reliable only when called directly or when called by try-with-resources.

Implementation Requirements:

Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use the `Cleaner` mechanism.

If this stream has an associated channel then this method will close the channel, which in turn will close this stream. Subclasses that override this method should be prepared to handle possible reentrant invocation.

Throws:

`IOException` - if an I/O error occurs.

getFD

```
public final FileDescriptor getFD()
                            throws IOException
```

Returns the `FileDescriptor` object that represents the connection to the actual file in the file system being used by this `FileInputStream`.

Returns:

the file descriptor object associated with this stream.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`FileDescriptor`

getChannel

```
public FileChannel getChannel()
```

Returns the unique `FileChannel` object associated with this file input stream.

The initial `position` of the returned channel will be equal to the number of bytes read from the file so far. Reading bytes from this stream will increment

the channel's position. Changing the channel's position, either explicitly or by reading, will change this stream's file position.

Returns:

the file channel associated with this file input stream

Since:

1.4

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.